Notes

Zeyuan Ye

April 12, 2026

Contents

1 Fisher information estimation **2**
1.1 Introduction . . . 2
1.2 Preliminary results . . . 3
1.2.1 Bayes' rule to express Fisher score in terms of Stein score . . . 3
1.2.2 Conditional denoising score matching . . . 3
1.2.3 Preliminary experiments . . . 4
1.3 Future possible extensions . . . 4
1.4 Method . . . 4

2 Estimating continuous-valued neural representational geometry with denoising score matching **7**
2.1 Introduction . . . 7
2.2 Methods . . . 7
2.3 Preliminary results . . . 8
2.4 Supplementary information: From dataset $\{(\theta_i, x_i)\}_{i=1}^N$ to the h-matrix . . . 9
2.5 Supplementary information: Hellinger distance and bounds on classification accuracy 12

3 Estimating Hellinger distance RDM using Flow Matching **13**
3.1 Toy dataset . . . 13
3.2 Ground truth RDM computation . . . 14
3.3 Denoising score matching does not work very well for high-dimensional dataset . . . 14
3.4 Flow matching: a short introduction . . . 16
3.5 Estimating the score from flow matching . . . 16
3.6 Flow matching works well for high-dimensional dataset . . . 17
3.7 Discussion: The next few important things . . . 19
3.8 Supplementary information: velocity network in flow matching. . . . 19
3.9 Supplementary information: cosine scheduler in flow matching. . . . 19
3.10 Supplementary information: conditional score under the Gaussian path . . . 19
3.11 Supplementary information: the marginal score as the expectation of the conditional score . . . 20

1 Fisher information estimation

1.1 Introduction

The brain represents the external world through internal neural representations. For example, while driving, our brain represents the surrounding spatial environment and the positions of nearby cars. Forming internal representations is a fundamental ability of the brain and serves as the basis for other cognitive functions.

This internal representation is recently referred to as a *cognitive map*. Importantly, a cognitive map is usually different from the corresponding physical map. In spatial navigation, for instance, the physical environment may be a square with a uniform spatial layout. However, the corresponding cognitive map need not be uniform: animals often form biased representations of locations associated with their own niches. The idea of a cognitive map is not limited to spatial locations. Another example is the neural representation of grating orientation. In the physical world, grating images with orientations θ and $\theta + \Delta\theta$ differ only by a rotation. If one measures the Euclidean distance between such images, that distance is independent of θ, reflecting the rotational symmetry of the stimulus space. In this sense, no orientation is intrinsically more special than another in the physical world. In contrast, animals often show better perceptual discrimination for vertical and horizontal orientations than for oblique ones. This suggests that the cognitive map of grating orientation differs from the physical map.

Fisher information is a fundamental quantity for characterizing such cognitive maps. At a high level, the Fisher information at a stimulus value θ quantifies the upper bound on how accurately θ can be decoded from neural activity. Therefore, Fisher information tells us how well the brain represents a particular stimulus value.

From another perspective, Fisher information measures the local distance between nearby stimulus values in neural representation space. Returning to the grating example, although the Euclidean distance between physical images with nearby orientations is independent of orientation, the neural distance—as quantified by Fisher information—can be larger near vertical and horizontal orientations than near oblique ones. This geometric perspective directly connects Fisher information to the notion of a cognitive map: Fisher information defines a local metric on the stimulus manifold represented by the neural population. An important advantage of Fisher information over the Euclidean distance is that Fisher information is invariant to the subjective choice of coordinate system. Fisher information therefore reflects intrinsic properties of the neural representation.

However, estimating Fisher information from neural population activity is hard, arguably because neural responses are both high-dimensional and noisy. Existing methods for estimating Fisher information can be broadly divided into two categories.

First, there are model-based methods, which assume a specific statistical model for neural activity, such as Poisson spiking or an exponential-family model. In practice, however, neural population activity is shaped by complex inter-neuron interactions, and the true statistical model is typically unknown.

Second, there are model-free methods. A central idea in this category is to train a classifier to distinguish between two nearby stimulus values based on neural activity x, and then use the classifier performance to estimate Fisher information. This approach is natural when stimulus values are discretized and repeated trials are available for each stimulus. However, many neuroscience experiments (such as navigation tasks)—especially increasingly common naturalistic experiments—do not provide repeated trials at discrete stimulus values; for example, an animal's spatial location varies continuously during navigation. To apply classical model-free methods in such settings, one must first discretize the continuous stimulus into bins, which introduces subjective choices such as

bin width. Moreover, such discretization fails to exploit the continuity of the stimulus space, where nearby observations may contain useful information for estimating Fisher information.

More recently, score-matching-based methods have been proposed for Fisher information estimation. Score matching is a generative framework closely related to diffusion models, which have achieved remarkable success in generating high-dimensional data such as images, audio, and video. Recent work has adapted score-based techniques to estimate Fisher information. However, these methods often require modifying the score matching method itself, such as latent-variable inference or simulator-specific objectives. As a result, they cannot always directly leverage the broad and rapidly developing family of existing score-matching methods. Furthermore, their applicability to high-dimensional, noisy neural data—especially discrete spiking data—remains unclear.

In this manuscript, we propose a simple alternative. Using Bayes' rule, we express the Fisher score in terms of Stein score which can be directly estimated by score-matching methods without any modification. In this way, our approach can immediately benefit from the broad and rapidly advancing score-matching literature. Moreover, the proposed method can be applied directly to spiking data, without converting spikes into continuous firing rates or imposing a Poisson assumption.

1.2 Preliminary results

1.2.1 Bayes' rule to express Fisher score in terms of Stein score

Score matching is a method to estimate the Stein score, however, is different from Fisher score. Therefore we express Fisher score in terms of Stein score using Bayes' rule.

$$\nabla_\theta \log p(x \mid \theta) = \nabla_\theta \log p(\theta \mid x) - \nabla_\theta \log p(\theta), \tag{1}$$

where $\theta \in \mathbb{R}^d$ is the stimulus parameter, such as grating orientation or spatial position; $x \in \mathbb{R}^n$ is the neural population activity; $\nabla_\theta \log p(x \mid \theta)$ is the Fisher score, $\nabla_\theta \log p(\theta \mid x)$ is the Stein score, and $\nabla_\theta \log p(\theta)$ is the score of the stimulus prior. In controlled experiments the prior may be known (e.g. uniform orientations); otherwise it can be learned from samples of θ alone. When $d \ll n$, the posterior score $\nabla_\theta \log p(\theta \mid x)$ can be estimated with conditional score matching; the prior score $\nabla_\theta \log p(\theta)$ can be estimated with a separate unconditional score model on θ.

1.2.2 Conditional denoising score matching

Here we focus on describing conditional denoising score matching for estimating $\nabla_\theta \log p(\theta \mid x)$. A typical way of using conditional denoising score matching is to let it learn and generate high-dimensional data; however, here we use it in an opposite way, the condition x is high-dimensional and the target θ is typically low-dimensional. Though, this does not change the mathematical nature of the problem – any conditional score matching method can be used to estimate $\nabla_\theta \log p(\theta \mid x)$. It is a plug-in component of our Fisher information estimation framework.

For better understanding, here we describe a concrete implementation of conditional denoising score matching. The spirit of denoising score matching is denoising. One adds Gaussian noise to a variable of interest and fits a network so that its output matches the noise that was injected; equivalently, the trained network approximates the Stein score—the gradient of the log-density—of a slightly smoothed distribution.

Let θ denote the stimulus parameter and let x denote an observation drawn from the conditional model $p(x \mid \theta)$. We sample a log-noise level $\log \sigma$ uniformly on an interval $[\log \sigma_{\min}, \log \sigma_{\max}]$, draw a standard normal scalar $z \sim \mathcal{N}(0, 1)$, and form the noisy latent $\tilde{\theta} = \theta + \sigma z$. We parameterize

the score by a neural network s_ϕ with trainable weights ϕ and train it by minimizing the noise-conditional denoising objective

$$\mathcal{L}(\phi) = \mathbb{E}\left[\left\|\sigma\, s_\phi(\tilde{\theta}, x, \sigma) + z\right\|^2\right], \tag{2}$$

where the expectation is taken over (θ, x) drawn from the generative process, over the random choice of σ, and over z. Here $\mathcal{L}(\phi)$ is the scalar loss to be minimized, ϕ collects all network parameters, and $s_\phi(\tilde{\theta}, x, \sigma)$ is the network output intended to approximate the score of a σ-smoothed conditional posterior, in the θ-direction, at $(\tilde{\theta}, x, \sigma)$: namely $\nabla_{\tilde{\theta}} \log p_\sigma(\tilde{\theta} \mid x)$, where $p_\sigma(\cdot \mid x)$ is the density of $\tilde{\theta} = \theta + \sigma z$. As $\sigma \to 0$, $p_\sigma(\cdot \mid x)$ converges to $p(\cdot \mid x)$ and $\nabla_{\tilde{\theta}} \log p_\sigma(\tilde{\theta} \mid x)$ converges to the Stein score $\nabla_\theta \log p(\theta \mid x)$. Together with $\nabla_\theta \log p(\theta)$, estimated by a second denoising score model on θ alone (see Method), we obtain the Fisher score via equation 1.

1.2.3 Preliminary experiments

We illustrate the full pipeline on a Gaussian toy model with scalar θ and two-dimensional observations $x \in \mathbb{R}^2$: cosine tuning means $\mu(\theta)$ and a θ-dependent covariance (heteroscedastic, correlated noise). We draw $n_{\text{total}} = 4000$ pairs with $\theta \sim \text{Uniform}[-3, 3]$, use a 70/30 train/eval split (2800/1200), and train (i) a conditional score network $s_\phi(\tilde{\theta}, x, \sigma)$ for the posterior score, (ii) an unconditional score network $s_\psi(\tilde{\theta}, \sigma)$ for the prior on θ, and (iii) local decoder baselines per θ bin. Figure 1 compares Fisher information vs. θ for analytic ground truth, posterior-only score at $\sigma_{\min}$, likelihood score $s_\phi - s_\psi$ at $\sigma_{\min}$, and the decoder (shaded bands: reported uncertainty).

Numerical metrics vs. ground truth (35 valid bins) are:

Method vs. GT	RMSE	MAE	Correlation
Posterior score ($\sigma_{\min}$)	8.525	5.489	0.918
Likelihood score ($s_{\text{post}} - s_{\text{prior}}$, $\sigma_{\min}$)	8.039	5.246	0.930
Decoder	21.417	13.738	0.727

1.3 Future possible extensions

Planned directions include comparisons on additional toy models with known ground truth (including spiking observations), and applications to neural data in visual and navigation tasks to characterize representation bias in cognitive-map terms.

1.4 Method

Data-generating model

The latent parameter is sampled as

$$\theta \sim \text{Uniform}[-3, 3],$$

and observations are generated from a Gaussian conditional family

$$x \mid \theta \sim \mathcal{N}\big(\mu(\theta), \Sigma(\theta)\big), \qquad x \in \mathbb{R}^2.$$

Each coordinate of $\mu(\theta)$ uses a cosine tuning curve with a fixed phase; the covariance $\Sigma(\theta)$ varies smoothly with θ (coordinate-wise standard deviations and correlation), yielding heteroscedastic correlated Gaussian observations. The experiment in Section 2 uses $n_{\text{total}} = 4000$, seed 7, and train fraction 0.7 (2800 train, 1200 eval).

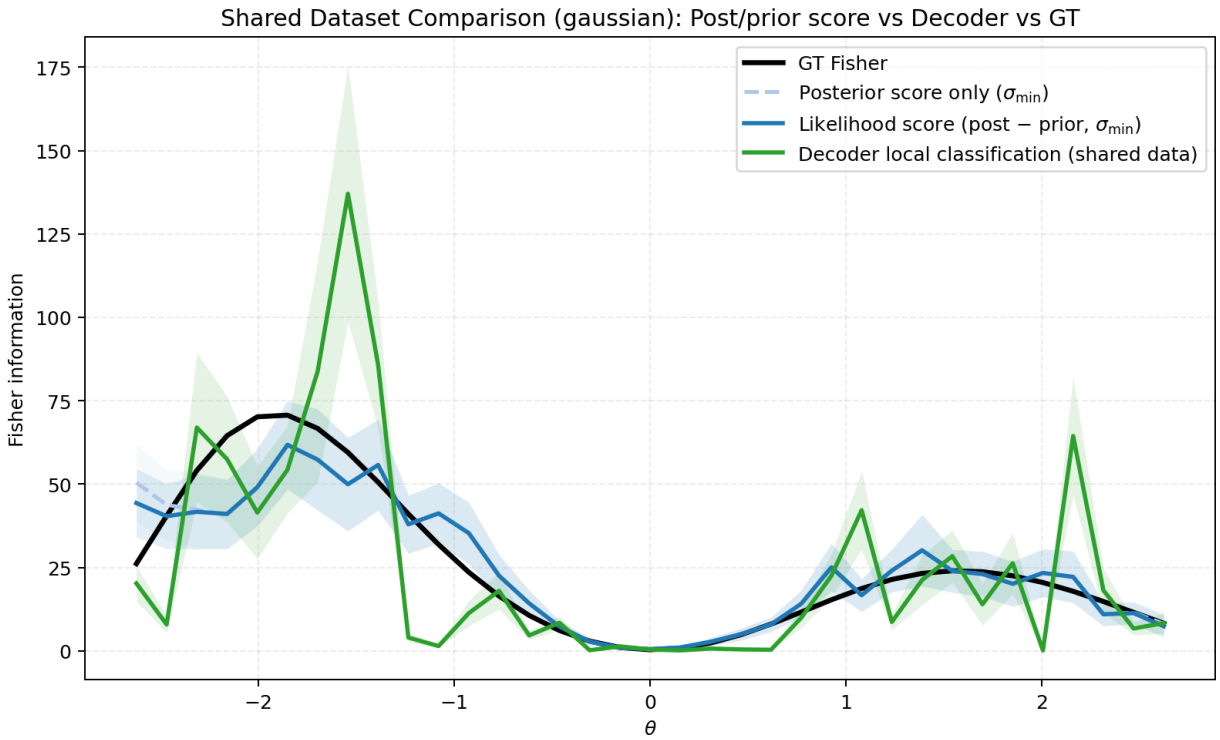

Figure 1: Gaussian toy ($n_{\text{total}} = 4000$, $x \in \mathbb{R}^2$). Fisher information vs. θ: ground truth (black), posterior score at $\sigma_{\min}$ (dashed blue), likelihood score $s_{\text{post}} - s_{\text{prior}}$ at $\sigma_{\min}$ (solid blue), local decoder (green).

Posterior and prior score models

Posterior (conditional) score. We train $s_\phi(\tilde{\theta}, x, \sigma)$ with continuous noise-conditional denoising score matching. For each pair (θ, x) from the data-generating process,

$$\log \sigma \sim \text{Uniform}(\log \sigma_{\min}, \log \sigma_{\max}), \qquad z \sim \mathcal{N}(0,1), \qquad \tilde{\theta} = \theta + \sigma z.$$

The objective is

$$\mathcal{L}(\phi) = \mathbb{E}\left[\left(\sigma s_\phi(\tilde{\theta}, x, \sigma) + z\right)^2\right].$$

Noise levels are tied to the empirical spread of θ on the score *fit* split: with $\hat{s}_\theta = \text{std}(\theta_{\text{fit}}) = 1.7273$,

$$\sigma_{\min} = 0.01\,\hat{s}_\theta = 0.01727, \qquad \sigma_{\max} = 0.25\,\hat{s}_\theta = 0.4318,$$

with 12 logarithmically spaced evaluation σ values between these extremes. Training uses a 15% validation holdout from the score-training pool (2380 fit / 420 validation), early stopping with patience 1000, minimum improvement 10^{-4}, exponential moving average (EMA) smoothing of the validation loss with $\alpha = 0.05$, and restoration of the best checkpoint.

Prior (unconditional) score. We train a separate network $s_\psi(\tilde{\theta}, \sigma)$ on θ samples only, with the same noise injection on $\tilde{\theta}$ and the denoising objective

$$\mathcal{L}(\psi) = \mathbb{E}\left[\left(\sigma s_\psi(\tilde{\theta}, \sigma) + z\right)^2\right],$$

and analogous early stopping (same patience, tolerance, EMA $\alpha = 0.05$). This approximates the prior score $\nabla_\theta \log p(\theta)$ in the smoothed sense used by DSM.

Score-based Fisher estimators

Let σ_* denote the smallest evaluation noise level (here 0.01727). For each evaluation pair (θ, x) we form posterior and prior score predictions at σ_* and define the likelihood score $\hat{s}_{\text{lik}} = s_\phi(\theta, x, \sigma_*) - s_\psi(\theta, \sigma_*)$. Fisher is estimated by binning θ into 35 equal-width bins on $[\theta_{\text{low}} + m, \theta_{\text{high}} - m]$ with margin $m = 0.30$, requiring at least 10 points per bin, and averaging squared scores within bins:

$$\widehat{I}_{\text{post}}(\theta_0) = \mathbb{E}\big[s_\phi(\theta, x, \sigma_*)^2 \,\big|\, \theta \in \text{bin}(\theta_0)\big]\,, \qquad \widehat{I}_{\text{lik}}(\theta_0) = \mathbb{E}\big[\hat{s}_{\text{lik}}(\theta, x)^2 \,\big|\, \theta \in \text{bin}(\theta_0)\big]\,.$$

The primary reported score curve is $\widehat{I}_{\text{lik}}$ (posterior minus prior); $\widehat{I}_{\text{post}}$ is kept as an ablation.

Decoder Fisher baseline

For each bin center θ_0, define two nearby parameters

$$\theta_+ = \theta_0 + \frac{\varepsilon}{2}, \qquad \theta_- = \theta_0 - \frac{\varepsilon}{2}, \qquad \varepsilon = 0.12.$$

A local classifier is trained to distinguish samples near θ_+ and θ_- using neighborhood bandwidth 0.10 in θ. If $\ell(x)$ is the classifier logit, then for small ε,

$$\ell(x) \approx \varepsilon\, \partial_\theta \log p(x \mid \theta_0),$$

and the decoder-based Fisher estimator is

$$\widehat{I}_{\text{dec}}(\theta_0) = \frac{1}{\varepsilon^2}\, \mathbb{E}\big[\ell(x)^2\big]\,.$$

Decoder training uses validation fraction 0.15, minimum validation class size 10, patience 100, minimum improvement 10^{-4}, EMA smoothing with $\alpha = 0.2$, up to 80 epochs, and best-epoch restoration.

Ground-truth Fisher and evaluation protocol

For the Gaussian conditional model, we evaluate ground truth analytically,

$$I_{\text{gt}}(\theta) = \mu'(\theta)^\top \Sigma(\theta)^{-1} \mu'(\theta) + \frac{1}{2}\, \text{tr}\big[\Sigma(\theta)^{-1} \Sigma'(\theta) \Sigma(\theta)^{-1} \Sigma'(\theta)\big]\,,$$

with derivatives induced by the chosen mean and covariance parameterization (equivalently the Fisher-trace definition).

Estimated curves are compared to I_{gt} over the 35 bins using RMSE, MAE, and Pearson correlation, as in Figure 1.

2 Estimating continuous-valued neural representational geometry with denoising score matching

2.1 Introduction

Representational similarity analysis is widely used in neuroscience and cognitive science to characterize how neural populations encode stimuli by estimating a representational dissimilarity matrix. Existing approaches have provided useful tools for this purpose, including correlation-based distances, Euclidean distances. However, these methods does not account for the noise structure of the data, and is variant to subjective choices of the coordinate system. Mahalanobis distances and decoding-based measures exploit the statistical nature of the problem. However, in order to estimate covariance or decoding, data must be binned first. Binning breaks the continuous nature of the problem, furthermore, when the number of data is sparse, binning reduce the number of data points even more, reducing the method reliability. Other methods like Wishart process does not binning the data yet requires explicitly assume the statistical model of the data (e.g. simple Gaussian) which is often to simplistic.

Here we propose a score-based continuous-stimulus method for estimating representational geometry directly from neural responses without binning the stimulus variable. Our method exploits the smooth structure of the stimulus axis by estimating the conditional score of the neural response distribution with respect to stimulus using score matching (a type of diffusion model), and then using this quantity to construct pairwise dissimilarities with a direct probabilistic interpretation. This provides a principled framework for recovering representational geometry of continuous-valued stimuli from sparse and noisy neural data.

2.2 Methods

Denoting the continuous-valued stimulus variable as θ, and the neural population response is modeled as a conditional distribution $p(x \mid \theta)$ which is unknown. A dataset is sampled from the joint distribution $p(x, \theta) = p(\theta)p(x \mid \theta)$ where $p(\theta)$ is the prior distribution which is often known by the experimenter (e.g. sampling uniform orientation of the grating image) or easy to estimate because of the low-dimensional nature of the stimulus.

Representation dissimilarity analysis cares about the "distance" between two stimulus conditions, mathematically, the distance between two conditional distributions. A straightforward choice is KL divergence, however, KL divergence is not a real distance metric. Therefore in this project, we use Hellinger distance $H(p_i, p_j)$

$$H^2(p_i, p_j) = 1 - \int \sqrt{p(x \mid \theta_i)\, p(x \mid \theta_j)}\, dx, \qquad p_i := p(x \mid \theta_i), \;\; p_j := p(x \mid \theta_j).$$

Besides being a true distance metric, Hellinger distance is attractive here for two additional reasons. First, it is locally connected to Fisher information. When two stimulus conditions are infinitesimally close, the squared Hellinger distance reduces, to leading order, to the Fisher-information metric. Second, Hellinger distance bounds the performance of the optimal binary classifier. In particular, a larger Hellinger distance implies a smaller achievable Bayes error. For these reasons, Hellinger distance provides a natural probabilistic notion of representational dissimilarity.

To connect Hellinger distance to a quantity that can be estimated from samples, define the log-likelihood ratio

$$\Delta\ell_{ij}(x) := \log p(x \mid \theta_j) - \log p(x \mid \theta_i).$$

Then the Bhattacharyya affinity can be written as

$$\sqrt{p(x \mid \theta_i)\, p(x \mid \theta_j)} = p(x \mid \theta_i) \exp\left(\frac{\Delta\ell_{ij}(x)}{2}\right),$$

which yields

$$H^2(p_i, p_j) = 1 - \mathbb{E}_{x\sim p_i}\left[\exp\left(\frac{\Delta\ell_{ij}(x)}{2}\right)\right].$$

Although exact, this one-sided form is asymmetric at the sample level because the expectation is taken only under p_i. To obtain a symmetric sample-wise quantity, we instead use the identity

$$H^2(p_i, p_j) = \frac{1}{2}\,\mathbb{E}_{x_i\sim p_i,\, x_j\sim p_j}\left[h_{ij}(x_i) + h_{ij}(x_j)\right],$$

where

$$h_{ij}(x) := 1 - \operatorname{sech}\left(\frac{\Delta\ell_{ij}(x)}{2}\right).$$

This form is explicitly symmetric in i and j, and expresses the Hellinger distance as the average of two sample-wise contributions, one evaluated on $x_i \sim p_i$ and the other on $x_j \sim p_j$.

The remaining challenge is therefore to estimate $\Delta\ell_{ij}(x)$ without discretizing θ. To exploit the continuous nature of the problem, we decompose the log-likelihood ratio into integration

$$\Delta\ell_{ij}(x) = \int_{\theta_i}^{\theta_j} \partial_\theta \log p(x \mid \theta)\, d\theta.$$

If we are able to estimate the Fisher score of all data points θ_i, integration can be computed numerically into summation.

How to estimate Fisher score? We use Bayes' rule:

$$\partial_\theta \log p(x \mid \theta) = \partial_\theta \log p(\theta \mid x) - \partial_\theta \log p(\theta).$$

The gradient of prior distribution can be computed analytically if the prior distribution is known by the experimentalist; if not known, estimating is relatively straightforward due to the low-dimensional nature of the stimulus.

We use denoising score matching to estimate $\partial_\theta \log p(\theta \mid x)$. The detail is written in previous section.

Overall, the pipeline is the following: Given a dataset $\{(\theta_i, x_i)\}_{i=1}^N$, we estimate the Fisher score of all data points θ_i using denoising score matching, and then perform path integration to obtain log-likelihood ratio, finally compute $h_{ij}(x)$. Notably, there's no data binning in this pipline. The final H_{ij} is the expecation of $h_{ij}(x)$ over all data points x_i and x_j, which can be estimated by simple binning, or more consistently using Gaussian process regression to estimate without any binning. But for simplicity, we use binning in preliminary results. The details of the pipeline is given in Supplementary Information.

2.3 Preliminary results

We generated a toy 2D linear model where neural tuning is $(x, 2x)$ with independent Gaussian noise std linearly ramped from 0.2 to 4. As an approximation of the ground truth, we generate large dataset with 5000 sample points and then estimated Hellinger distance matrix and pairwise decoding matrix, respectively. Hellinger distance matrix is estimated as stated in the method section; pairwise decoding matrix is estimated by binning the data and use logistic regression.

To study how well each of these two methods performs when the dataset size is small, we subsampled smaller dataset with $n = 50$, sample points. For each of the subsampled dataset, we estimated RDM respectively and then compared the correlation of the off-diagonal entries to the approximated ground truth RDM.

Results showing that our Hellinger distance matrix estimated by denoising score matching method exploit the continuous nature of the problem and converges faster than pairwise decoding method, see Figure 2.

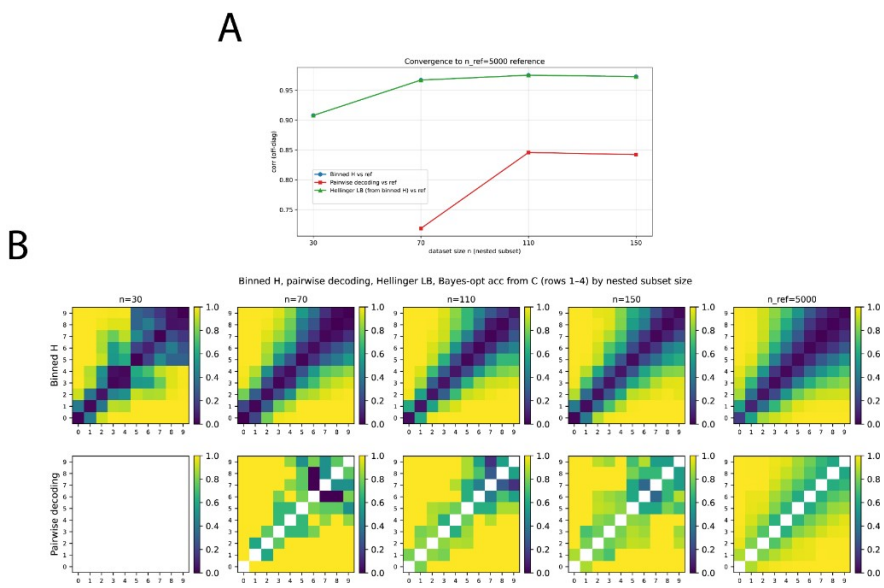

Figure 2: Estimation from subset of dataset convergence of geometry estimates to a large reference ($n_{\text{ref}} = 5000$). **(A)** Correlation of off-diagonal matrix entries to the reference for binned Hellinger (H), pairwise decoding, and a Hellinger lower bound derived from binned H (blue and green overlap), as nested subset size n increases. **(B)** Heatmaps of binned H, pairwise decoding, Hellinger lower bound, and Bayes-optimal accuracy (rows) across $n \in \{30, 70, 110, 150\}$ and the reference column. Pairwise decoding was computed by binning the data and using logistic regression to estimate decoding accuracy. Dataset: linear tuning $(x, 2x)$ with Gaussian noise std linearly ramped from 0.2 to 4.

2.4 Supplementary information: From dataset $\{(\theta_i, x_i)\}_{i=1}^N$ to the h-matrix

We start from a dataset

$$\mathcal{D} = \{(\theta_i, x_i)\}_{i=1}^N,$$

where $\theta_i \in \mathbb{R}$ is a scalar condition and x_i is the corresponding observation. Our goal is to construct an $N \times N$ matrix

$$\mathbf{H}^2 = [H_{ij}^2],$$

whose entry H_{ij}^2 is the sample-wise symmetric contribution of x_i when comparing the conditional distributions $p(x \mid \theta_i)$ and $p(x \mid \theta_j)$.

Step 1: learn posterior and prior scores in θ. Using denoising score matching, we train two score models:

$$s_\phi(\tilde{\theta}, x, \sigma) \approx \partial_\theta \log p_\sigma(\tilde{\theta} \mid x), \qquad r_\psi(\tilde{\theta}, \sigma) \approx \partial_\theta \log p_\sigma(\tilde{\theta}),$$

where p_σ denotes the distribution smoothed in θ by Gaussian noise of scale σ. Concretely, the posterior score model is trained by minimizing

$$\mathcal{L}_{\text{post}}(\phi) = \mathbb{E}_{(\theta,x)\sim\mathcal{D},\ \sigma,\ z\sim\mathcal{N}(0,1)}\Big[\big\|\sigma s_\phi(\theta+\sigma z, x, \sigma) + z\big\|^2\Big],$$

and the prior score model is trained by minimizing

$$\mathcal{L}_{\text{prior}}(\psi) = \mathbb{E}_{\theta\sim\{\theta_i\},\ \sigma,\ z\sim\mathcal{N}(0,1)}\Big[\big\|\sigma r_\psi(\theta+\sigma z, \sigma) + z\big\|^2\Big].$$

For a small evaluation noise level σ_{eval}, define the recovered Fisher score

$$\hat{g}(\theta, x) := s_\phi(\theta, x, \sigma_{\text{eval}}) - r_\psi(\theta, \sigma_{\text{eval}}).$$

By Bayes' rule,

$$\partial_\theta \log p(x \mid \theta) = \partial_\theta \log p(\theta \mid x) - \partial_\theta \log p(\theta),$$

so $\hat{g}(\theta, x)$ approximates the Fisher score $\partial_\theta \log p(x \mid \theta)$.

Step 2: sort the dataset by θ. For notational simplicity, we assume the samples are reordered such that

$$\theta_1 \le \theta_2 \le \cdots \le \theta_N.$$

If the original dataset is not sorted, we sort it by θ, perform all computations below in the sorted order, and permute the final matrices back to the original order if desired.

Step 3: construct the recovered Fisher score matrix. For each sample x_i and each observed condition θ_k, define

$$G_{ik} := \hat{g}(\theta_k, x_i).$$

Thus $\mathbf{G} = [G_{ik}] \in \mathbb{R}^{N\times N}$ is the matrix whose i-th row corresponds to a fixed observation x_i, and whose k-th column evaluates the recovered Fisher score at condition θ_k.

Step 4: construct the cumulative integral matrix. For each row i, define the cumulative trapezoidal integral

$$C_{i1} = 0, \qquad C_{i,k+1} = C_{ik} + \frac{\theta_{k+1}-\theta_k}{2}\big(G_{ik} + G_{i,k+1}\big), \quad k = 1, \ldots, N-1.$$

Equivalently,

$$C_{ij} = \sum_{k=1}^{j-1} \frac{\theta_{k+1}-\theta_k}{2}\big(G_{ik} + G_{i,k+1}\big), \qquad j = 1, \ldots, N.$$

For fixed i, C_{ij} approximates the integral

$$C_{ij} \approx \int_{\theta_1}^{\theta_j} \hat{g}(\theta, x_i)\, d\theta.$$

Step 5: construct the log-likelihood-ratio matrix $\Delta\mathbf{L}$. For a sample (θ_i, x_i), define

$$\Delta\ell_{ij}(x_i) := \log p(x_i \mid \theta_j) - \log p(x_i \mid \theta_i).$$

Since

$$\Delta\ell_{ij}(x_i) = \int_{\theta_i}^{\theta_j} \partial_\theta \log p(x_i \mid \theta)\, d\theta,$$

we estimate it by

$$\Delta L_{ij} := C_{ij} - C_{ii}.$$

Thus the matrix $\Delta\mathbf{L} = [\Delta L_{ij}] \in \mathbb{R}^{N\times N}$ is the estimated log-likelihood-ratio matrix, where

$$\Delta L_{ij} \approx \Delta\ell_{ij}(x_i).$$

By construction,

$$\Delta L_{ii} = 0.$$

Step 6: construct the directed sample-wise contribution matrix. Define

$$H_{ij}^{\rightarrow} := 1 - \operatorname{sech}\left(\frac{\Delta L_{ij}}{2}\right).$$

This is a directed matrix, because its (i, j)-entry is evaluated on x_i only.

Step 7: symmetrize the matrix. To obtain a symmetric pairwise matrix, average the two directions:

$$H_{ij}^{sym} := \frac{1}{2}\left(H_{ij}^{\rightarrow} + H_{ji}^{\rightarrow}\right).$$

Thus the full pipeline is

$$\{(\theta_i, x_i)\}_{i=1}^N \longrightarrow s_\phi,\, r_\psi \longrightarrow \mathbf{G} \longrightarrow \mathbf{C} \longrightarrow \Delta\mathbf{L} \longrightarrow \mathbf{H}^{\rightarrow} \longrightarrow \mathbf{H}^{sym}.$$

Algorithmic summary.

1. Train $s_\phi(\theta, x, \sigma)$ and $r_\psi(\theta, \sigma)$.
2. Compute
$$G_{ik} = s_\phi(\theta_k, x_i, \sigma_{\text{eval}}) - r_\psi(\theta_k, \sigma_{\text{eval}})$$
for all i, k.
3. Compute the cumulative integral matrix $\mathbf{C}$ row by row:
$$C_{i1} = 0, \qquad C_{i,k+1} = C_{ik} + \frac{\theta_{k+1} - \theta_k}{2}(G_{ik} + G_{i,k+1}).$$
4. Compute the directed log-ratio matrix
$$\Delta L_{ij} = C_{ij} - C_{ii}.$$
5. Compute the directed contribution matrix
$$H_{ij}^{\rightarrow} = 1 - \operatorname{sech}\left(\frac{\Delta L_{ij}}{2}\right).$$

6. Symmetrize:

$$H_{ij}^{sym} = \frac{1}{2}\left(H_{ij}^{\rightarrow} + H_{ji}^{\rightarrow}\right).$$

Finally, the Hellinger distance matrix is given by the expectation which can be computed by binning H_{ij}^{sym} or regression method:

$$\mathbf{H}^2 = [\mathbb{E}_{x_i \sim p_i,\, x_j \sim p_j}[H_{ij}^{sym}]].$$

2.5 Supplementary information: Hellinger distance and bounds on classification accuracy

For binary classification between conditions i and j with equal class priors, let

$$p_i(x) = p(x \mid \theta_i), \qquad p_j(x) = p(x \mid \theta_j),$$

and let H_{ij} denote the Hellinger distance between p_i and p_j. The Bayes-optimal classification accuracy is

$$A_{ij}^* = \frac{1}{2}\big(1 + \mathrm{TV}(p_i, p_j)\big),$$

where $\mathrm{TV}(p_i, p_j)$ is the total variation distance. Using the inequalities

$$H_{ij}^2 \le \mathrm{TV}(p_i, p_j) \le \sqrt{2H_{ij}^2 - H_{ij}^4},$$

we obtain

$$\frac{1}{2}(1 + H_{ij}^2) \le A_{ij}^* \le \frac{1}{2}\left(1 + \sqrt{2H_{ij}^2 - H_{ij}^4}\right).$$

Therefore, a measured Hellinger distance directly constrains the best achievable pairwise classification accuracy. In particular, larger Hellinger distance implies better possible discriminability, with $H_{ij} = 0$ corresponding to chance-level accuracy $A_{ij}^* = 1/2$, and $H_{ij} = 1$ corresponding to perfect classification $A_{ij}^* = 1$.

3 Estimating Hellinger distance RDM using Flow Matching

In previous section, we showed that denoising score matching method is able to estimate the Hellinger distance well (Figure 2). The dataset we used is, however, a simple line manifold in a two dimensional space. Here we test our method using a high dimensional toy dataset.

3.1 Toy dataset

We use a synthetic conditional model for a scalar stimulus θ and a multivariate observation $x \in \mathbb{R}^d$ with $d \geq 2$. Stimuli are drawn independently from a uniform density on a bounded interval,

$$\theta \sim \mathrm{Uniform}(\theta_{\mathrm{low}}, \theta_{\mathrm{high}}). \tag{3}$$

For each neuron $j \in \{1, \ldots, d\}$ we fix a *tuning center* θ_j by placing d equally spaced points on $[\theta_{\mathrm{low}}, \theta_{\mathrm{high}}]$ (endpoints included).

Random bump amplitudes. Before drawing joint samples (θ, x), we sample one amplitude per neuron,

$$a_j \sim \mathrm{Uniform}(a_{\mathrm{low}}, a_{\mathrm{high}}), \qquad j = 1, \ldots, d, \tag{4}$$

independently across j. The vector $(a_1, \ldots, a_d)$ is then held fixed.

Conditional mean (tuning curve). Given θ, the mean of x_j is a Gaussian bump in θ centered at θ_j:

$$\mu_j(\theta) = a_j \exp\!\big(-\kappa \left(\omega\,(\theta - \theta_j)\right)^2\big), \qquad j = 1, \ldots, d, \tag{5}$$

with bump precision $\kappa \geq 0$ and stimulus-axis scale ω. The conditional mean vector is $\mu(\theta) = (\mu_1(\theta), \ldots, \mu_d(\theta))^{\mathsf{T}}$.

Observation noise. Conditionally on θ, the observation is multivariate Gaussian with diagonal covariance. Let $\sigma > 0$ be a constant baseline noise scale shared across all neurons, and let α be a fixed activity–variance coupling strength (the arithmetic mean of two nonnegative hyperparameters in the implementation). The per-neuron conditional variances are

$$v_j(\theta) = d\,\sigma^2 \left(1 + \alpha\,|\mu_j(\theta)|\right), \qquad j = 1, \ldots, d, \tag{6}$$

with independence across neurons,

$$x \mid \theta \sim \mathcal{N}\big(\mu(\theta),\, \mathrm{diag}(v_1(\theta), \ldots, v_d(\theta))\big). \tag{7}$$

Thus noise magnitude increases with the absolute tuned response $|\mu_j(\theta)|$, and each neuron's noise standard deviation scales proportionally to $\sqrt{d}$ for fixed σ, α, and $\mu_j(\theta)$. The leading factor d ties per-neuron variance to the population size d: as more neurons are observed, each carries proportionally larger variance so that increasing dimension alone does not drive the joint observation toward an effectively noiseless, trivially informative regime for θ.

3.2 Ground truth RDM computation

Hellinger distance. We discretize the stimulus range into B bins. For bins i, j, write $p_i(x) = p(x \mid \bar{\theta}_i)$ and $p_j(x) = p(x \mid \bar{\theta}_j)$ for the conditional observation density implied by the toy generative model (Gaussian mean $\mu(\theta)$ and diagonal variances $v_j(\theta)$ as above). We estimate the squared Hellinger H_{ij}^2 using the one-sided identity:

$$H_{ij}^2 = 1 - \mathbb{E}_{x \sim p_i}\left[\exp\left(\tfrac{1}{2}\left(\log p_j(x) - \log p_i(x)\right)\right)\right], \tag{8}$$

and the expectation is replaced by a sample mean. Specifically, draw m samples from $p(\cdot \mid \bar{\theta}_i)$ for each row i, evaluate $\log p(x \mid \bar{\theta}_i)$ and $\log p(x \mid \bar{\theta}_j)$ under the closed-form Gaussian likelihood, and average the exponentials (implemented with a stabilized log-mean-exp). The result is clipped to $[0, 1]$. The squared-root of this $B \times B$ matrix is the ground truth Hellinger distance matrix. Non-diagonal entries are used for computing correlation with the estimated RDM.

Pairwise decoding. The GT of pairwise decoding is estimated by binning the data and using logistic regression to estimate decoding accuracy with large $n_{\text{ref}} = 5000$ datapoints.

3.3 Denoising score matching does not work very well for high-dimensional dataset

In two dimensions, the learned Hellinger RDM tracks the Monte Carlo target closely as the nested training size n increases, outperforming the binning-based decoding baseline (Figure 3). At $d = 100$, however, neither the DSM-based Hellinger estimate nor the binned decoding matrix matches to approximated GT well at comparable n. I also tried 50D example, the result is similar, both method does not work very well.

In particular, for $d = 100$ the DSM estimate exhibits a false signal of artificially low squared region in the middle of the matrix, which raises concerns for practical use of the method in high dimension.

I tried to add hacks to DSM method but failed. Therefore I decided to switch using flow matching estimating the fisher score which according to my experience, flow matching is a more robust, easier to use method.

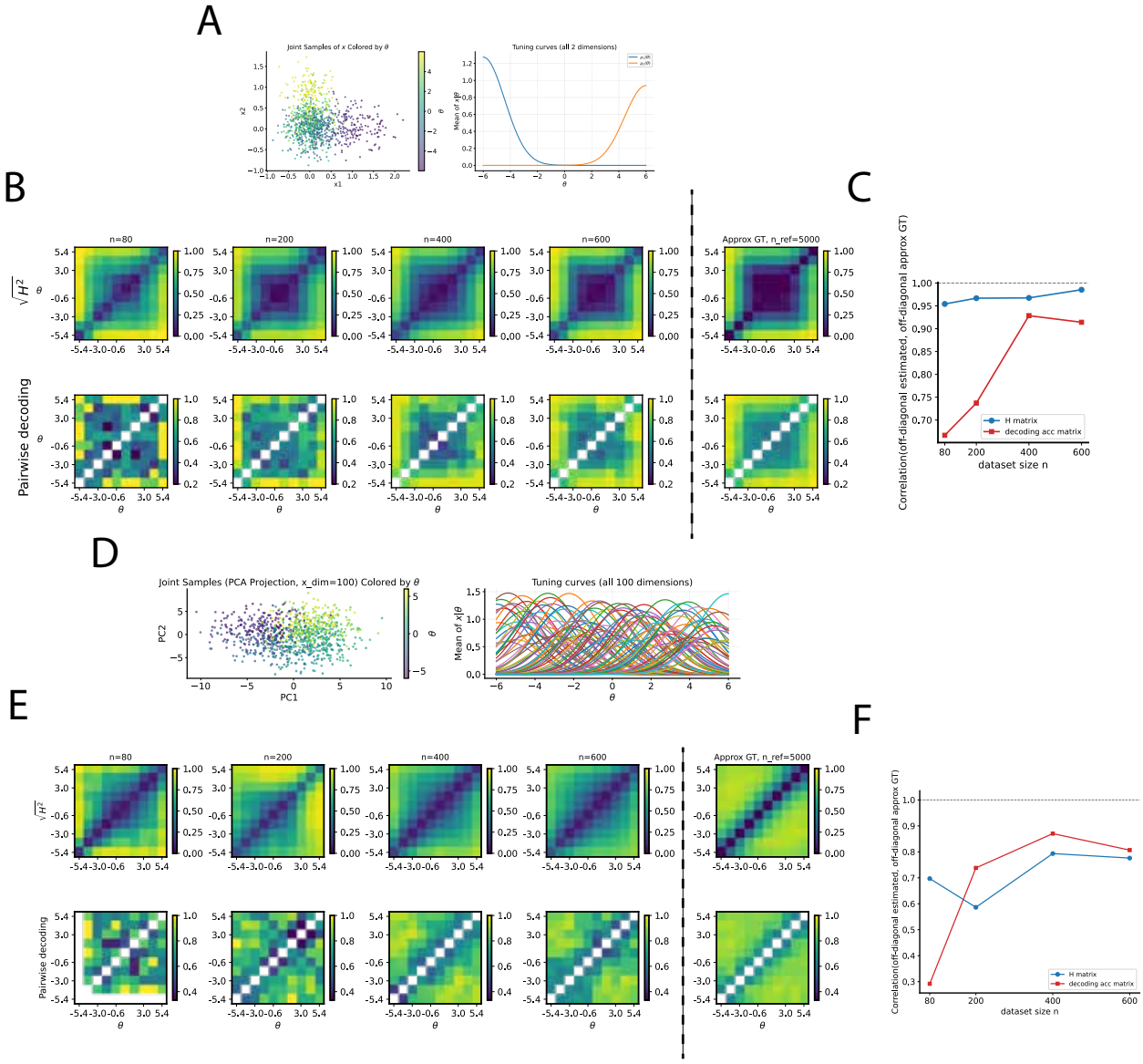

Figure 3: DSM method does not perform well in high dimensional dataset. (A) 2D toy dataset. (B) Estimated RDM with different dataset size. GT of Hellinger distance was estimated by Monte Carlo sampling, while GT decoding acc matrix was estimated by decoding large dataset (5000). (C) Correlation from estimation to approximated GT.(D, E, F) Same as (A, B, C) but for 100D dataset. Note that dsm estimation of Hellinger distance matrix yield robust false signal across different dataset size: a large low-distance square region in the middle.

3.4 Flow matching: a short introduction

Flow matching learns a time-dependent vector field that transports samples from a simple base distribution to a target distribution by integrating an ordinary differential equation (ODE). Let $t \in [0,1]$ and let p_0 denote a tractable reference on $\mathbb{R}^d$ (for example, a standard Gaussian). The goal is to construct a velocity field $v_\phi(x,t)$ such that trajectories solving

$$\frac{\mathrm{d}}{\mathrm{d}t} x_t = v_\phi(x_t, t), \qquad x_0 \sim p_0, \tag{9}$$

induce x_1 to be close to a target p_1 (in our case, the model distribution or a conditional $p_1(\cdot \mid \theta)$).

To obtain the regression objective of velocity field, one needs to first define a probability path from x_0 to x_1. A concrete construction is affine path. Fix scalar functions $\alpha, \beta : [0,1] \to \mathbb{R}$ with $\alpha(0) = 1$, $\beta(0) = 0$, $\alpha(1) = 0$, and $\beta(1) = 1$, and define the affine path

$$x_t = \alpha(t)\, x_0 + \beta(t)\, x_1, \tag{10}$$

so that $x_t = x_0$ at $t = 0$ and $x_t = x_1$ at $t = 1$. Holding (x_0, x_1) fixed, the induced velocity is

$$u_t := \frac{\mathrm{d}}{\mathrm{d}t} x_t = \alpha'(t)\, x_0 + \beta'(t)\, x_1. \tag{11}$$

Given t sampled uniformly on $[0,1]$, a simple Monte Carlo flow-matching objective matches the network prediction to this bridge velocity:

$$\mathcal{L}_{\mathrm{FM}}(\phi) = \mathbb{E}_{t \sim \mathcal{U}[0,1]}\, \mathbb{E}_{x_0 \sim p_0,\, x_1 \sim p_1} \Big[\big\| v_\phi(x_t, t) - u_t \big\|_2^2 \Big]. \tag{12}$$

Minimizing $\mathcal{L}_{\mathrm{FM}}$ encourages v_ϕ to realize the same time evolution as the chosen coupling, so integrating the learned ODE from p_0 pushes mass toward p_1.

Conditional flow matching. In the setting of this note, the target is conditional on the stimulus θ. One trains $v_\phi(x, t \mid \theta)$ by replacing p_1 with $p(\cdot \mid \theta)$ in the expectation over x_1:

$$\mathcal{L}_{\mathrm{CFM}}(\phi) = \mathbb{E}_\theta\, \mathbb{E}_{t \sim \mathcal{U}[0,1]}\, \mathbb{E}_{x_0 \sim p_0,\, x_1 \sim p(\cdot \mid \theta)} \Big[\big\| v_\phi(x_t, t \mid \theta) - u_t \big\|_2^2 \Big], \tag{13}$$

with x_t as in (10) and u_t as in (11) along each sampled bridge (we use the same symbols for both objectives). In this note, we used affine path with cosine scheduler and used a MLP to parameterize the velocity field, see Supplementary information.

3.5 Estimating the score from flow matching

The result of this subection is included in Yaron Lipman *et al.*, *Flow Matching Guide and Code* (preprint, 2024), `https://doi.org/10.48550/arXiv.2412.06264` (equation 4.78). The following is a self-contained derivation.

Denoising score matching learns the score function $\nabla_x \log p(x)$ directly (or a noisy version of it). Flow matching, by contrast, learns a velocity field. For Gaussian probability paths, these two quantities are closely related: the score can be recovered from the velocity by a simple analytic transformation.

Consider the Gaussian path

$$x_t = \beta(t)\, x_1 + \alpha(t)\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{14}$$

where $\alpha(t)$ and $\beta(t)$ are scalar schedules. This Gaussian path is the affine path (10) when the base draw x_0 is isotropic normal. Conditionally on x_1, this defines a Gaussian perturbation of the data. Averaging over x_1 gives a noisy marginal density $p_t(x_t)$. Our ultimate goal is the data score $\nabla_x \log p(x)$, but when t is close to 1, the noisy density p_t is close to the data density p, so it is often enough to estimate the noisy score

$$s_t(x_t) := \nabla_{x_t} \log p_t(x_t),$$

which converges to the data score as $t \to 1$.

The key observation is that, under the Gaussian path, the conditional score has a very simple form (see Subsection 3.10):

$$s_t(x_t \mid x_1) = -\frac{x_t - \beta(t)x_1}{\alpha(t)^2} = -\frac{\varepsilon}{\alpha(t)}. \tag{15}$$

Taking conditional expectation over x_1 given x_t yields

$$s_t(x_t) = -\frac{1}{\alpha(t)}\,\mathbb{E}[\varepsilon \mid x_t]. \tag{16}$$

Therefore, estimating the noisy score reduces to estimating the posterior mean of the (unknown) Gaussian noise.

To connect this quantity to flow matching, we differentiate the path (14) with respect to time. This gives the conditional velocity

$$u_t = \dot{\beta}(t)\,x_1 + \dot{\alpha}(t)\,\varepsilon.$$

Now combine this expression with the path equation itself. Since both involve the same unknowns, x_1 and ε, we can eliminate x_1 and solve for ε in terms of x_t and u_t. This gives

$$\varepsilon = \frac{u_t - \frac{\dot{\beta}(t)}{\beta(t)}x_t}{\dot{\alpha}(t) - \alpha(t)\frac{\dot{\beta}(t)}{\beta(t)}}. \tag{17}$$

Finally, take conditional expectation given x_t. If we define the marginal velocity field by

$$v_t(x_t) := \mathbb{E}[u_t \mid x_t],$$

then (17) immediately implies

$$\mathbb{E}[\varepsilon \mid x_t] = \frac{v_t(x_t) - \frac{\dot{\beta}(t)}{\beta(t)}x_t}{\dot{\alpha}(t) - \alpha(t)\frac{\dot{\beta}(t)}{\beta(t)}}.$$

Substituting this into (16), we obtain the noisy score in terms of the velocity field:

$$s_t(x_t) = \frac{\frac{\dot{\beta}(t)}{\beta(t)}x_t - v_t(x_t)}{\alpha(t)\left(\dot{\alpha}(t) - \alpha(t)\frac{\dot{\beta}(t)}{\beta(t)}\right)}. \tag{18}$$

Finally, a well-trained flow matching velocity network $v_\phi(x_t, t)$ learns the marginal velocity field $v_t(x_t)$. By replacing v_t with $v_\phi(x_t, t)$, we are able to compute the noisy score $s_t(x_t)$ which is an approximation of the data score. Since x_1 is given in the data rather than x_t, therefore the score we in practice computed is $s_t(x_1)$ where we fix $t = 0.8$ in this note.

3.6 Flow matching works well for high-dimensional dataset

See Figure 4.

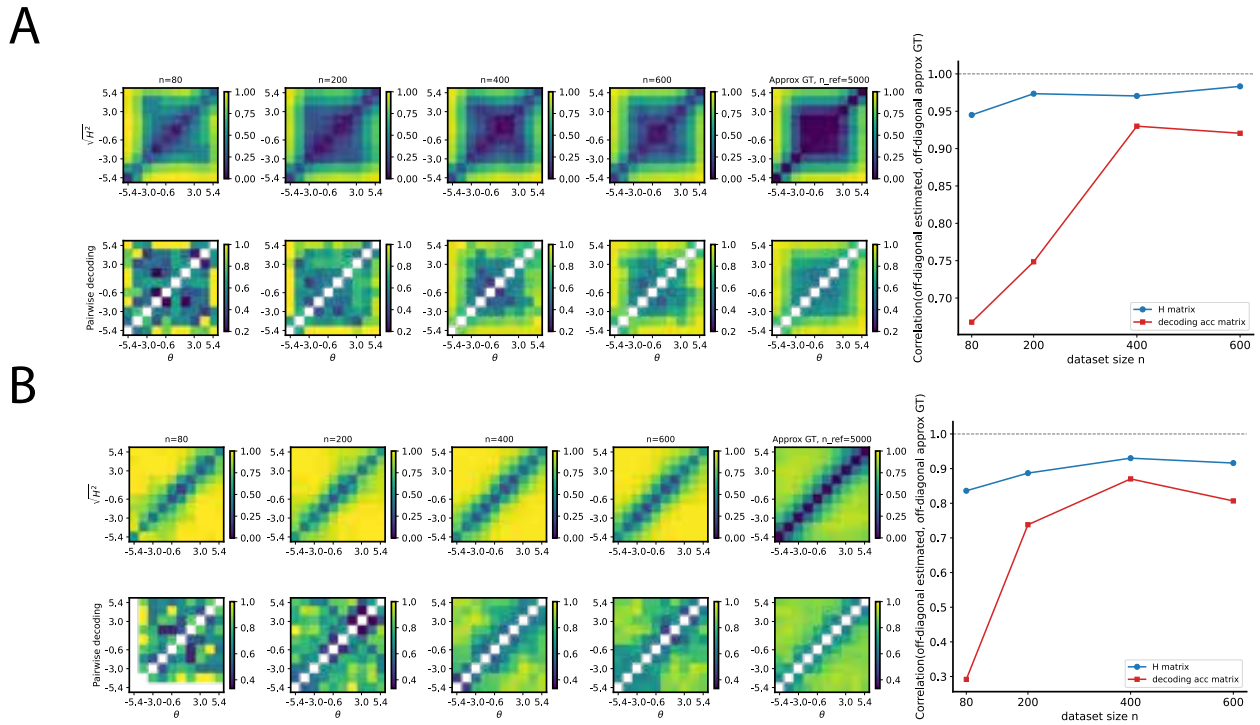

Figure 4: Flow matching for Fisher score estimation in the random-amplitude Gaussian toy model (same protocol as Figure 3). Estimated Hellinger RDMs and decoding matrices stay much closer to the Monte Carlo and high-n decoding ground truths at $d = 100$ than DSM, without the spurious low-distance block in the RDM middle as seen in Figure 3.

3.7 Discussion: The next few important things

1. What is the effect of t value? How to choose it?
2. Can we interpret the choice of t value as adding noise to data and measures the robustness of representation to noise attack?
3. How to estimate the RDM estimation uncertainty empirically or theoretically?
4. Path integration may fail for periodic manifold since path integration makes "detours".
5. Whether the current method works well for discrete label?

3.8 Supplementary information: velocity network in flow matching.

In the experiments reported here, the conditional (posterior) velocity is parameterized as a fully connected network (MLP): the input is $(\theta_t, x, \psi(t)) \in \mathbb{R}^{1+d+1}$, where θ_t is the scalar state along the flow bridge, $x \in \mathbb{R}^d$ is the observation, and $\psi(t) = \log t - \log(1-t)$ is a logit-time feature (with t restricted to a compact subinterval of $(0,1)$ so that ψ is finite). The map consists of three hidden layers of width 128 with SiLU activations, followed by a linear readout to a scalar velocity. The prior velocity uses the same MLP pattern on $(\theta_t, \psi(t)) \in \mathbb{R}^2$ with hidden width 128 and three hidden layers.

3.9 Supplementary information: cosine scheduler in flow matching.

With $t \in [0,1]$, the cosine choice for the affine path (10) is

$$\alpha(t) = \cos\Big(\frac{\pi}{2}\, t\Big), \qquad \beta(t) = \sin\Big(\frac{\pi}{2}\, t\Big). \tag{19}$$

3.10 Supplementary information: conditional score under the Gaussian path

We consider the Gaussian probability path

$$x_t = \beta(t)\, x_1 + \alpha(t)\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{20}$$

Conditioned on x_1, the randomness comes only from ε, so $x_t \mid x_1$ is Gaussian with mean $\beta(t)x_1$ and covariance $\alpha(t)^2 I$:

$$p_t(x_t \mid x_1) = \mathcal{N}\big(x_t; \beta(t)x_1, \alpha(t)^2 I\big). \tag{21}$$

Therefore,

$$\log p_t(x_t \mid x_1) = -\frac{d}{2}\log\big(2\pi\alpha(t)^2\big) - \frac{1}{2\alpha(t)^2}\, \|x_t - \beta(t)x_1\|^2, \tag{22}$$

where d is the dimension of x_t. Since the first term does not depend on x_t, differentiating with respect to x_t gives

$$\nabla_{x_t} \log p_t(x_t \mid x_1) = -\frac{1}{2\alpha(t)^2}\, \nabla_{x_t}\|x_t - \beta(t)x_1\|^2. \tag{23}$$

Using the identity

$$\nabla_x \|x - a\|^2 = 2(x - a),$$

we obtain

$$s_t(x_t \mid x_1) := \nabla_{x_t} \log p_t(x_t \mid x_1) = -\frac{x_t - \beta(t)x_1}{\alpha(t)^2}. \tag{24}$$

Finally, from the path definition (20),

$$x_t - \beta(t)x_1 = \alpha(t)\varepsilon.$$

Substituting this into (24) yields

$$s_t(x_t \mid x_1) = -\frac{\alpha(t)\varepsilon}{\alpha(t)^2} = -\frac{\varepsilon}{\alpha(t)}. \tag{25}$$

Thus, under the Gaussian path, the conditional score has a particularly simple interpretation: it is just the negative latent noise, rescaled by the noise level $\alpha(t)$.

3.11 Supplementary information: the marginal score as the expectation of the conditional score

We now explain why the noisy marginal score

$$s_t(x_t) := \nabla_{x_t} \log p_t(x_t)$$

can be written as the conditional expectation of the conditional score

$$s_t(x_t \mid x_1) := \nabla_{x_t} \log p_t(x_t \mid x_1).$$

Recall that the noisy marginal density is obtained by averaging the conditional density over the data distribution:

$$p_t(x_t) = \int p_t(x_t \mid x_1)\, p(x_1)\, dx_1. \tag{26}$$

Differentiating both sides with respect to x_t gives

$$\nabla_{x_t} p_t(x_t) = \int \nabla_{x_t} p_t(x_t \mid x_1)\, p(x_1)\, dx_1. \tag{27}$$

Using the identity

$$\nabla_{x_t} p_t(x_t \mid x_1) = p_t(x_t \mid x_1)\, \nabla_{x_t} \log p_t(x_t \mid x_1),$$

we obtain

$$\nabla_{x_t} p_t(x_t) = \int p_t(x_t \mid x_1)\, p(x_1)\, \nabla_{x_t} \log p_t(x_t \mid x_1)\, dx_1. \tag{28}$$

Now divide both sides by $p_t(x_t)$. By definition of the score,

$$s_t(x_t) = \nabla_{x_t} \log p_t(x_t) = \frac{1}{p_t(x_t)} \int p_t(x_t \mid x_1)\, p(x_1)\, \nabla_{x_t} \log p_t(x_t \mid x_1)\, dx_1. \tag{29}$$

The factor

$$\frac{p_t(x_t \mid x_1)\, p(x_1)}{p_t(x_t)}$$

is exactly the posterior density $p(x_1 \mid x_t)$. Therefore,

$$s_t(x_t) = \int p(x_1 \mid x_t)\, s_t(x_t \mid x_1)\, dx_1 = \mathbb{E}[s_t(x_t \mid x_1) \mid x_t]\,. \tag{30}$$